

# TechHub — Git & GitHub Governance

**Document:** 01 — Git & GitHub Best Practices **Version:** 1.0.0 **Owner:** All Members · Standards  
Enforced by Member 3 **Status:** Active

---

## Table of Contents

1. [Repository Structure](#)
  2. [Branching Strategy](#)
  3. [Commit Message Convention](#)
  4. [Pull Request Rules](#)
  5. [GitHub Actions CI/CD Ownership](#)
  6. [Standard CI/CD Pipeline Template](#)
  7. [CI/CD Pipeline Stage Reference](#)
  8. [GitHub Projects Tracking Board](#)
  9. [Git Anti-Patterns](#)
- 

## 1. Repository Structure

TechHub uses a **monorepo** — all services live in one repository. Each service has its own folder. CI pipelines are path-filtered to only trigger on relevant changes.

```

techhub/                                ← Monorepo root
|
├─ .github/
|   └─ workflows/
|       ├── gateway.yml                ← Member 1
|       ├── user-service.yml           ← Member 1
|       ├── event-service.yml          ← Member 2
|       ├── project-service.yml        ← Member 2
|       ├── team-service.yml           ← Member 3
|       ├── notification-service.yml   ← Member 3
|       ├── community-service.yml      ← Member 4
|       ├── frontend.yml               ← Member 4
|       └─ integration-tests.yml       ← Member 3 (coordinates)
|
├─ gateway/                            ← Member 1
├─ user-service/                       ← Member 1
├─ event-service/                      ← Member 2
├─ project-service/                   ← Member 2
├─ team-service/                      ← Member 3
├─ notification-service/              ← Member 3
├─ community-service/                 ← Member 4
├─ frontend/                          ← Member 4
|
├─ k8s/                               ← Kubernetes manifests (see DevOps doc)
├─ docker-compose.yml                 ← Member 3 coordinates · all contribute
|
└─ docs/
    └─ sre/
        ├── 01_GIT_AND_GITHUB_GOVERNANCE.md    ← this file
        ├── 02_DEVOPS_K8S_DOCKER_GOVERNANCE.md
        └─ 03_SRE_STANDARDS_OPERATIONS.md

```

## 2. Branching Strategy

### 2.1 Branch Model

```

main          → Production-ready code.
               → Protected. Requires PR + 2 approvals + all CI green.

develop       → Integration branch.
               → All feature branches merge here first.
               → Requires PR + 1 approval.

feature/*     → Short-lived feature development.

```

```
fix/*           → Bug fix branches.
infra/*         → Infrastructure, Docker, Kubernetes, CI changes.
release/*       → Release preparation (if required).
```

## 2.2 Branch Naming Convention

```
feature/<member>/<short-description>
fix/<member>/<short-description>
infra/<member>/<short-description>
```

Examples:

```
feature/m1/user-jwt-authentication
feature/m2/event-creation-endpoint
infra/m3/kafka-kubernetes-statefulset
fix/m4/community-null-pointer-on-post
```

**|** `<member>` = `m1`, `m2`, `m3`, or `m4`. `<short-description>` = kebab-case, maximum 5 words.

## 2.3 Merge Flow

```
feature/*  —PR (1 reviewer)→  develop  —PR (2 reviewers)→  main
```

## 2.4 Branch Lifecycle Rules

- Delete feature branches immediately after they are merged.
- Never reuse branch names.
- Never commit directly to `main` or `develop` — always go through a PR.
- Branches must be kept up to date with `develop` before merging.

---

## 3. Commit Message Convention

TechHub follows the **Conventional Commits** specification.

### 3.1 Format

```
<type>(<scope>): <short imperative description>
```

```
[optional body — explain WHY, not WHAT]
```

```
[optional footer — references: Closes #123]
```

### 3.2 Commit Types

| Type                  | When to Use                                          |
|-----------------------|------------------------------------------------------|
| <code>feat</code>     | New feature or endpoint                              |
| <code>fix</code>      | Bug fix                                              |
| <code>chore</code>    | Build tooling, dependency update, no behavior change |
| <code>docs</code>     | Documentation changes                                |
| <code>refactor</code> | Code restructure with no behavior change             |
| <code>test</code>     | Adding or fixing tests                               |
| <code>ci</code>       | GitHub Actions workflow changes                      |
| <code>infra</code>    | Docker, Kubernetes, infrastructure changes           |
| <code>style</code>    | Formatting, whitespace (no logic change)             |

### 3.3 Scope Reference

| Scope                             | Maps To              |
|-----------------------------------|----------------------|
| <code>gateway</code>              | API Gateway service  |
| <code>user-service</code>         | User Service         |
| <code>event-service</code>        | Event Service        |
| <code>project-service</code>      | Project Service      |
| <code>team-service</code>         | Team Service         |
| <code>notification-service</code> | Notification Service |
| <code>community-service</code>    | Community Service    |
| <code>frontend</code>             | Next.js frontend     |
| <code>k8s</code>                  | Kubernetes manifests |
| <code>compose</code>              | Docker Compose files |
| <code>kafka</code>                | Kafka configuration  |

## Scope

## Maps To

postgres

Database configuration

redis

Redis configuration

### 3.4 Good Commit Examples

```
bash

feat(user-service): add JWT token refresh endpoint

fix(gateway): correct upstream routing for /api/events

infra(k8s): add resource limits to notification-service deployment

ci(event-service): add integration test stage to GitHub Actions pipeline

test(project-service): add Testcontainers integration tests for project creation

docs(sre): update port allocation registry with community service

chore(user-service): upgrade Spring Boot to 3.2.1
```

### 3.5 Bad Commit Examples — Rejected in Code Review

```
bash

# ❌ Too vague
fix stuff
update
wip
changes

# ❌ Wrong format
Added JWT to user service
fixed the bug in gateway
KAFKA WORKS NOW

# ❌ Too large – should be split into multiple commits
feat: implement all services and deploy to kubernetes
```

### 3.6 Commit Frequency Rules

- Commit early and often — at least one commit per logical change.
- Never bundle multiple unrelated changes in one commit.

- Never commit broken code to `develop` or `main`.
- A commit to `develop` must pass CI before other members pull it.

## 4. Pull Request Rules

### 4.1 PR Requirements by Target Branch

| Requirement            | <code>develop</code> | <code>main</code> |
|------------------------|----------------------|-------------------|
| Minimum reviewers      | 1                    | 2                 |
| CI checks must pass    | ✓                    | ✓                 |
| No merge conflicts     | ✓                    | ✓                 |
| Coverage not decreased | ✓                    | ✓                 |
| PR description filled  | ✓                    | ✓                 |
| Linked ticket/task     | ✓                    | ✓                 |
| Self-approval          | ✗ Never              | ✗ Never           |

### 4.2 PR Description Template

Every PR must use this structure:

markdown

### ## What does this PR do?

Brief description of the change and its purpose.

### ## Why?

The motivation or the problem it solves.

### ## Related Task

Closes #<issue-number>

### ## Type of Change

- [ ] New feature
- [ ] Bug fix
- [ ] Infrastructure / DevOps
- [ ] Documentation
- [ ] Refactor

### ## Testing Done

- [ ] Unit tests pass locally
- [ ] Integration tests pass locally
- [ ] Manually tested in Docker Compose

### ## Checklist

- [ ] No hardcoded values (localhost, passwords, ports)
- [ ] No secrets in code or config
- [ ] Swagger annotations updated (if API changed)
- [ ] K8s manifests updated (if deployment changed)
- [ ] .env.example updated (if new env vars added)

## 4.3 Reviewer Checklist

Reviewers must verify before approving:

- ☐ Code follows naming and coding conventions
- ☐ No hardcoded `localhost`, IPs, passwords, or tokens
- ☐ Environment variables are externalized
- ☐ Dockerfile follows standards (if applicable)
- ☐ K8s manifests follow standards (if applicable)
- ☐ Tests are present and meaningful
- ☐ Swagger annotations are present on new endpoints
- ☐ No secrets or credentials in any file
- ☐ CI pipeline passes

4.4 PR Size Limits

| PR Size   | Lines Changed  | Policy                |
|-----------|----------------|-----------------------|
| Small     | < 200 lines    | ✔ Preferred           |
| Medium    | 200–500 lines  | ✔ Acceptable          |
| Large     | 500–1000 lines | ⚠ Needs justification |
| Too Large | > 1000 lines   | ✖ Must be split       |

Large PRs are harder to review and harder to revert. Split your work into small, focused PRs whenever possible.

4.5 PR Anti-Patterns — Auto-Rejected

- PR contains commented-out dead code
- PR disables or removes tests
- PR introduces hardcoded credentials or localhost references
- PR does not have a description
- PR author approves their own PR
- PR mixes unrelated concerns (e.g., feature + infrastructure change in one PR)

4.6 Review Turnaround SLA

| Priority                           | Expected Review Time |
|------------------------------------|----------------------|
| Blocking (unblocks another member) | 2 hours              |
| Standard                           | Same day             |
| Documentation only                 | Next day             |

If a reviewer cannot review within the SLA, they must communicate in the team channel so another member can step in.

5. GitHub Actions CI/CD Ownership

5.1 Workflow File Ownership

| Workflow File            | Owner (Maintains) | Reviewer (Reviews PRs) |
|--------------------------|-------------------|------------------------|
| <code>gateway.yml</code> | Member 1          | Member 3               |



| Workflow File            | Owner (Maintains) | Reviewer (Reviews PRs) |
|--------------------------|-------------------|------------------------|
| user-service.yml         | Member 1          | Member 3               |
| event-service.yml        | Member 2          | Member 3               |
| project-service.yml      | Member 2          | Member 3               |
| team-service.yml         | Member 3          | Member 1               |
| notification-service.yml | Member 3          | Member 1               |
| community-service.yml    | Member 4          | Member 3               |
| frontend.yml             | Member 4          | Member 3               |
| integration-tests.yml    | Member 3          | All members            |

### 5.2 Pipeline Trigger Strategy

Each workflow is path-filtered to only run when its service changes:

```
yaml
on:
  push:
    branches: [main, develop]
    paths:
      - 'user-service/**'      # Only triggers when user-service/ changes
  pull_request:
    branches: [main, develop]
    paths:
      - 'user-service/**'
```

This prevents unrelated pipelines from running on every commit.

### 5.3 Required GitHub Repository Secrets

| Secret Name     | Description             | Set By   |
|-----------------|-------------------------|----------|
| DOCKER_USERNAME | Docker Hub username     | Member 3 |
| DOCKER_PASSWORD | Docker Hub access token | Member 3 |
| DB_PASSWORD     | PostgreSQL password     | Member 3 |
| JWT_SECRET      | JWT signing secret      | Member 1 |

All secrets are set once by the designated member in **Settings → Secrets and variables → Actions**.

## 5.4 Branch Protection Rules (Configure on GitHub)

For `main`:

- Require pull request before merging
- Require 2 approving reviews
- Dismiss stale pull request approvals when new commits are pushed
- Require status checks to pass before merging
- Require branches to be up to date before merging
- Include administrators (no bypass)

For `develop`:

- Require pull request before merging
- Require 1 approving review
- Require status checks to pass before merging

---

## 6. Standard CI/CD Pipeline Template

Every service's workflow must follow this 5-stage structure. Copy and adapt this template:

```
yaml
```

```
# .github/workflows/<service-name>.yaml
name: <Service Name> CI/CD

on:
  push:
    branches: [main, develop]
    paths:
      - '<service-folder>/**'
      - '.github/workflows/<service-name>.yaml'
  pull_request:
    branches: [main, develop]
    paths:
      - '<service-folder>/**'

env:
  SERVICE_NAME: <service-name>
  SERVICE_DIR: ./<service-folder>
  IMAGE_NAME: techhub/<service-name>
  JAVA_VERSION: '21'

jobs:

  # — STAGE 1: Build —————
  build:
    name: 🛠️ Build
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Set up JDK ${{ env.JAVA_VERSION }}
        uses: actions/setup-java@v4
        with:
          java-version: ${{ env.JAVA_VERSION }}
          distribution: 'temurin'
          cache: maven

      - name: Compile with Maven
        working-directory: ${{ env.SERVICE_DIR }}
        run: mvn clean compile -B

  # — STAGE 2: Test —————
  test:
    name: 🧪 Test
    runs-on: ubuntu-latest
    needs: build
```

```
steps:
  - uses: actions/checkout@v4

  - uses: actions/setup-java@v4
    with:
      java-version: ${ env.JAVA_VERSION }
      distribution: 'temurin'
      cache: maven

  - name: Run Unit Tests
    working-directory: ${ env.SERVICE_DIR }
    run: mvn test -B

  - name: Run Integration Tests
    working-directory: ${ env.SERVICE_DIR }
    run: mvn verify -P integration-tests -B

  - name: Upload Test Report
    if: always()
    uses: actions/upload-artifact@v4
    with:
      name: test-report-${ env.SERVICE_NAME }
      path: ${ env.SERVICE_DIR }/target/surefire-reports/
```

# — STAGE 3+4: Docker Build & Push —————

```
docker:
  name: 🐳 Docker Build & Push
  runs-on: ubuntu-latest
  needs: test
  if: github.ref == 'refs/heads/main' || github.ref == 'refs/heads/develop'
  steps:
    - uses: actions/checkout@v4

    - name: Build Docker Image
      run: |
        docker build \
          -t ${ env.IMAGE_NAME }:${ github.sha } \
          -t ${ env.IMAGE_NAME }:latest \
          ${ env.SERVICE_DIR }

    - name: Login to Docker Hub
      run: |
        echo "${ secrets.DOCKER_PASSWORD }" | \
        docker login -u "${ secrets.DOCKER_USERNAME }" --password-stdin

    - name: Push Docker Image
      run: |
```

```
docker push ${env.IMAGE_NAME}:${github.sha}
docker push ${env.IMAGE_NAME}:latest
```

# — STAGE 5: Deploy to Kubernetes —

deploy:

name: 🚀 Deploy to Kubernetes

runs-on: ubuntu-latest

needs: docker

if: github.ref == 'refs/heads/main'

steps:

- uses: actions/checkout@v4

- name: Set up kubectl

  - uses: azure/setup-kubectl@v3

- name: Update Image Tag in Deployment

  - run: |

    - sed -i "s|image: \${env.IMAGE\_NAME}.\*|image: \${env.IMAGE\_NAME}:\${github.sha}|" k8s/\${env.SERVICE\_NAME}/deployment.yaml

- name: Apply Kubernetes Manifests

  - run: |

    - kubectl apply -f k8s/\${env.SERVICE\_NAME}/ -n techhub

- name: Verify Rollout

  - run: |

    - kubectl rollout status deployment/\${env.SERVICE\_NAME} \
    - n techhub --timeout=120s

# — Failure Notification —

notify-failure:

name: ❌ Notify on Failure

runs-on: ubuntu-latest

needs: [build, test, docker]

if: failure()

steps:

- name: Log Failure

  - run: |

    - echo "Pipeline FAILED for \${env.SERVICE\_NAME}"

    - echo "Branch: \${github.ref}"

    - echo "Commit: \${github.sha}"

    - echo "Author: \${github.actor}"

## 7. CI/CD Pipeline Stage Reference

### 7.1 Stage Summary

| Stage               | Runs On           | Condition                                        | Output                   |
|---------------------|-------------------|--------------------------------------------------|--------------------------|
| Build               | Every push + PR   | Always                                           | Compiled code            |
| Test                | After build       | Always                                           | Test report artifact     |
| Docker Build & Push | After tests pass  | <code>main</code> or <code>develop</code> branch | Docker image in registry |
| Deploy              | After Docker push | <code>main</code> branch only                    | Running pod in K8s       |
| Notify Failure      | After any failure | On failure                                       | Log entry                |

### 7.2 Pipeline Status Badge

Add this to your service's `README.md`:

markdown

```
![[CI Status]](https://github.com/<org>/techhub/actions/workflows/<service-name>.yml/b
```

### 7.3 Common Pipeline Failures and Fixes

| Failure                                            | Likely Cause                | Fix                                                                       |
|----------------------------------------------------|-----------------------------|---------------------------------------------------------------------------|
| <code>mvn: command not found</code>                | Java not set up             | Add <code>setup-java</code> step                                          |
| <code>Cannot connect to Docker daemon</code>       | Docker not available        | Use <code>ubuntu-latest</code> runner                                     |
| <code>unauthorized: authentication required</code> | Docker secret missing/wrong | Check <code>DOCKER_USERNAME</code> / <code>DOCKER_PASSWORD</code> secrets |
| <code>kubect1: command not found</code>            | kubect1 not installed       | Add <code>azure/setup-kubect1</code> step                                 |
| <code>ImagePullBackOff</code> in K8s               | Wrong image name or tag     | Verify <code>IMAGE_NAME</code> env var matches Docker Hub repo            |
| Tests fail in CI but pass locally                  | Hardcoded localhost         | Replace with environment variable or Testcontainers                       |

## 8. GitHub Projects Tracking Board

### 8.1 Board Setup

Create a GitHub Project board with the following columns. Every task (Issue) moves left to right.

| Column       | Definition                                                               | Who Moves Cards Here |
|--------------|--------------------------------------------------------------------------|----------------------|
| Backlog      | All planned tasks, not yet started                                       | Member 3 / team      |
| To Do        | Ready to work on this sprint                                             | Task owner           |
| In Progress  | Actively being worked on                                                 | Task owner           |
| Code Review  | PR open, waiting for review                                              | Task owner           |
| Docker Ready | Service containerized + Compose working                                  | Task owner           |
| K8s Ready    | Service deployed to Kubernetes                                           | Task owner           |
| Integrated   | Service communicates with its dependencies                               | Task owner           |
| Tested       | Unit + integration tests pass                                            | Task owner           |
| Done         | Merged to <code>develop</code> or <code>main</code> , deployed, verified | Reviewer             |

### 8.2 Issue Template

Every GitHub Issue should include:

markdown

### ## Description

What needs to be done and why.

### ## Acceptance Criteria

- [ ] Criterion 1
- [ ] Criterion 2

### ## Assigned To

@member

### ## Service

e.g., user-service

### ## Definition of Done

- [ ] Code implemented
- [ ] Tests written and passing
- [ ] Docker image builds
- [ ] K8s manifest updated
- [ ] PR merged

## 8.3 Label System

| Label                | Color    | Meaning                       |
|----------------------|----------|-------------------------------|
| service:gateway      | Blue     | API Gateway                   |
| service:user         | Blue     | User Service                  |
| service:event        | Green    | Event Service                 |
| service:project      | Green    | Project Service               |
| service:team         | Purple   | Team Service                  |
| service:notification | Purple   | Notification Service          |
| service:community    | Orange   | Community Service             |
| infra                | Red      | Infrastructure / K8s / Docker |
| ci-cd                | Yellow   | GitHub Actions                |
| bug                  | Red      | Bug fix                       |
| blocked              | Dark red | Blocked on another task       |
| documentation        | Gray     | Docs only                     |



---

## 9. Git Anti-Patterns

The following practices are **explicitly forbidden** in TechHub and will result in PR rejection or mandatory rework.

### 9.1 Commit Anti-Patterns

| Anti-Pattern                                                           | Why It's Harmful               | Correct Practice                                             |
|------------------------------------------------------------------------|--------------------------------|--------------------------------------------------------------|
| Committing to <code>main</code> directly                               | Bypasses review and CI         | Always use a feature branch and PR                           |
| Giant commits (1000+ lines)                                            | Impossible to review or revert | Make small, focused commits                                  |
| Vague commit messages ( <code>fix stuff</code> ), ( <code>wip</code> ) | No traceability                | Follow Conventional Commits format                           |
| Committing broken code                                                 | Breaks CI for everyone         | Always compile and test locally first                        |
| Committing secrets or passwords                                        | Critical security risk         | Use env vars and Kubernetes Secrets                          |
| Committing <code>.env</code> files                                     | Leaks credentials              | Add <code>.env</code> to <code>.gitignore</code> immediately |
| Force-pushing to shared branches                                       | Destroys teammates' history    | Never force-push <code>main</code> or <code>develop</code>   |

### 9.2 Mandatory `.gitignore` Entries

Every service folder must have a `.gitignore` containing at minimum:

```
# Build output
target/
*.jar
*.war

# Environment files – NEVER commit these
.env
.env.local
.env.*.local

# IDE files
.idea/
*.iml
.vscode/
*.DS_Store

# Logs
*.log
logs/

# Node (frontend)
node_modules/
.next/
```